# “Linux IPC: Pipes & FIFOs”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## BACHELOR OF TECHNOLOGY

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINNERING**

**By**


| | |
|---|---|
| **K SAMANTH** | **2520080011** |
| **SURAJ.P** | **2520090179** |
| **V.PAVAN KUMAR** | **2520090079** |


Under the Esteemed Guidance of


**RAGHUPATHI**
**Assistant Professor**
**Department of Artificial intelligence & Data Structures**






**Koneru Lakshmaiah Education Foundation**
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University**
**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL**




# Declaration

The Project Report entitled "Linux IPC: Pipes & FIFOs" is a record of Bonafide work of **SAMANTH.K- 2520080011 , SURAJ .2520090179,V.PAVAN KUMAR - 2520090079** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.


| | |
|---|---|
| **K.SAMANTH** | **2520080011** |
| **SURAJ.P** | **2520090179** |
| **V. PAVAN KUMAR** | **2520090079** |

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**




# CERTIFICATE

This is certify that the mini project based report entitled "Linux IPC: Pipes & FIFOs" is a bonafide work done and submitted by **SAMANTH.K– 2520080011 ,**
in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University),
during the academic year **2024-2025.**

**Signature of the Guide**

**Signature of the Course Coordinator** **Signature of the HOD**

# ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Raghupathi**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. I. Sreenath (AI&DS)**, Head of the ***Department of artificial intelligence and data structures*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

| | |
|---|---|
| **SAMANTH.K** | **2520080011** |
| **SURAJ.P** | **2520090179** |
| **V.PAVAN KUMAR** | **2520090079** |

**Abstract**

Inter-Process Communication (IPC) is a mechanism that allows processes to exchange data and coordinate their activities. In Linux, Pipes and FIFOs are simple and efficient IPC mechanisms used for communication between processes. A Pipe provides a unidirectional communication channel, generally between related processes such as a parent and child process. A FIFO (First-In, First-Out), also called a named pipe, provides communication between processes that do not necessarily have a parent-child relationship. This project demonstrates how Linux Pipes and FIFOs can be used to transfer data between processes using system calls and file operations

# Index


| S. No. | Chapter | Topics / Subtopics | Page No. |
|---|---|---|---|
| | Acknowledgement | | i |
| | Abstract | | ii |
| 1 | Introduction | | 1 –5 |
| | | 1.1 Background of the Project | |
| | | 1.2 Problem Statement | |
| | | 1.3 Scope of the Project | |
| | | 1.4 Objectives of the Project | |
| | | 1.5 Importance of the Application | |
| | | 1.6 Target Users / Audience | |
| 2 | OS Concepts and Background Study | | 6–12 |
| | | 2.1 Operating System Concepts Related to the Project | |
| | | 2.2 Existing Methods / Algorithms | |
| | | 2.3 Proposed Approach | |
| | | 2.4 Advantages of the Proposed Approach | |
| 3 | System Requirements | | 13–17 |
| | | 3.1 Hardware Requirements | |
| | | 3.2 Software Requirements | |
| | | 3.3 Operating System Environment – Ubuntu / WSL | |
| | | 3.4 Development Tools and Compilers | |
| 4 | Technology and Tools Used | | 18–22 |
| | | 4.1 C / C++ Programming | |

| | | | |
|---|---|---|---|
| | | **4.2 GCC Compiler** | |
| | | **4.3 Linux System Calls** | |
| | | **4.4 POSIX Threads / Synchronization** | |
| | | **4.5 Linux Commands and Utilities** | |
| | | **4.6 Other Tools / Libraries Used** | |
| **5** | **System Architecture and Design** | | **23–29** |
| | | **5.1 Overall System Architecture** | |
| | | **5.2System Workflow** | |
| | | **5.3 Flowchart / Data Flow** | |
| | | **5.4 Module Description** | |

| | | | |
|---|---|---|---|
| | | **5.5 Input and Output Design** | |
| **6** | **Implementation** | | **30–45** |
| | | **6.1 Module-wise Implementation** | |
| | | **6.2 Algorithm / Pseudocode** | |
| | | **6.3 Core C / C++ Implementation** | |
| | | **6.4 Linux System Calls / APIs Used** | |
| | | **6.5 Process / Thread Implementation** | |
| | | **6.6 IPC / Synchronization Implementation** | |
| | | **6.7 Memory Management Implementation** | |
| | | **6.8 Optional User Interface Implementation** | |
| **7** | **Execution and Demonstration** | | **62–67** |
| | | **7.1 Compilation in Ubuntu** | |
| | | **7.2 Execution Commands** | |
| | | **7.3 Sample Inputs and Outputs** | |
| | | **7.4 Screenshots of Execution** | |
| | | **7.5 Demonstration of OS Concepts** | |
| **8** | **Conclusion** | | **79–83** |

| | | | |
|---|---|---|---|
| | | **8.1 Summary of the Project** | |
| | | **8.2 Objectives Achieved** | |
| | | **8.3 Results Achieved** | |
| | | **8.4 Skills and Learning Outcomes** | |
| **9** | **References** | | **84–86** |
| | | **9.1 Books** | |
| | | **9.2 Research Papers** | |
| | | **9.3 Linux / GNU Documentation** | |
| | | **9.4 Online Resources** | |


## CHAPTER – 1

## INTRODUCTION

### 1.1 Background of the Project

An operating system manages the execution of programs and provides mechanisms for processes to communicate with one another. In Linux, a process is an instance of a running program. A system may contain multiple processes executing independently, and managing their activities is an important responsibility of the operating system.

Linux provides signals as a mechanism for notifying processes about events and requesting specific actions. Signals can be used to interrupt, pause, resume, or terminate processes, depending on the signal and how the process handles it.

This project, Linux Signal-Based Process Communication and Control System, demonstrates the practical use of Linux signals to manage multiple processes. A controller process creates worker processes and sends them signals to control their activities. Worker processes respond through appropriate signal-handling behavior, while the controller monitors their status and coordinates their execution.

The project connects theoretical operating system concepts with practical system programming using the C language and Linux system calls.

### 1.2 Problem Statement

Managing multiple processes requires a reliable mechanism for controlling their execution and coordinating their activities. Without proper process management and communication, it can be difficult to monitor process states, send control commands, and ensure that processes terminate correctly.

The objective of this project is to develop a Linux-based system that allows a controller process to create, communicate with, monitor, and control multiple worker processes using signals. It demonstrates how Linux system calls and signal-handling mechanisms can be used to manage process activities and synchronize process termination.

### 1.3 Scope of the Project

The project focuses on process creation, signal-based communication, process control, and orderly termination in a Linux environment. It uses the C programming language and Linux system calls to demonstrate communication between a controller and worker processes.

Creating worker processes using fork().

Sending signals to selected processes using kill().

Handling supported signals using appropriate signal-handling mechanisms.

Observing process identifiers and execution status.

Coordinating child-process termination using wait() or waitpid().

Compiling, executing, and testing the program in Ubuntu/Linux.

The project is an educational prototype. Signals provide event notifications and control requests, rather than a general-purpose channel for transferring large data messages.

### 1.4 Objectives of the Project

To understand Linux process management and process creation.

To implement communication and control using Linux signals.

To demonstrate the use of fork(), kill(), getpid(), sigaction(), wait(), and waitpid(), as applicable to the implementation.

To manage multiple worker processes through a controller process.

To understand process states, signal handling, and synchronization.

To test the application in an Ubuntu/Linux environment.

### 1.5 Importance of the Application

The project is important because it demonstrates how an operating system supports communication and coordination between processes. It helps students understand the relationship between system calls and process behavior. The implementation also provides practical experience in C programming, debugging, Linux commands, and process lifecycle management.

The concepts demonstrated are relevant to system utilities, process supervisors, background services, and other applications that need to notify or control running processes.

### 1.6 Target Users / Audience

The primary audience includes students studying Operating Systems, system programming, and Linux fundamentals. The project can also be used by instructors to demonstrate process creation, signal delivery, process control, and child-process synchronization. It is intended as a learning prototype rather than a production process-management utility.

# CHAPTER – 2

## OS CONCEPTS AND BACKGROUND STUDY

### 2.1 Operating System Concepts Related to the Project

An operating system is system software that manages computer hardware, executes programs, allocates resources, and provides services to application programs. Linux is an open-source operating system that supports multitasking, process management, inter-process communication, and synchronization. These features make Linux suitable for developing system-level applications.

The Linux Signal-Based Process Communication and Control System demonstrates important operating system concepts using the C programming language and Linux system calls.

#### 2.1.1 Process Management

A process is an instance of a program that is currently executing. Linux allows multiple processes to run concurrently and provides mechanisms to create, manage, monitor, and terminate them. In this project, a controller process creates and manages multiple worker processes. Each worker performs its operations independently and responds to signals sent by the controller.

#### 2.1.2 Process Creation Using fork()

The fork() system call creates a new process called a child process. The process that calls fork() is known as the parent process. The return value of fork() helps distinguish the parent from the child and identify process-creation errors. In this project, the controller uses fork() to create worker processes that can be managed individually.

#### 2.1.3 Inter-Process Communication (IPC)

Inter-Process Communication (IPC) refers to mechanisms that allow separate processes to exchange information or coordinate their activities. Linux supports pipes, message queues, shared memory, sockets, and signals. This project uses signals as its primary mechanism for communication and process control. Signals notify worker processes about events or request actions such as pausing, resuming, or terminating execution.

#### 2.1.4 Linux Signals

A signal is a software notification delivered to a process to indicate that an event has occurred or that an action is requested. Common Linux signals include:

SIGINT: commonly generated when Ctrl+C is pressed in a terminal.
SIGTERM: requests a process to terminate.
SIGKILL: terminates a process and cannot be caught or ignored.
SIGSTOP: stops a process and cannot be caught or ignored.
SIGCONT: resumes a stopped process.
SIGUSR1 and SIGUSR2: user-defined signals that can be assigned application-specific purposes.
The controller sends appropriate signals, and workers respond according to their configured behavior.

### 2.1.5 Signal Handling

Signal handling is the mechanism through which a process responds to signals. Linux allows programs to define signal handlers for many signals using sigaction(). For reliable implementation, handlers should perform only safe, limited operations; more complex processing can be performed in the main execution loop.

### 2.1.6 Process Identification

Every Linux process has a process identifier (PID) while it exists. getpid() returns the PID of the calling process, while getppid() returns the PID of its parent. The kill() system call can send a signal to a process identified by its PID. Despite its name, kill() does not always terminate a process; its effect depends on the signal supplied.

### 2.1.7 Process Synchronization and Termination

Process synchronization coordinates related processes and helps ensure that process completion is handled correctly. Linux provides wait() and waitpid() for a parent process to wait for child processes to change state, including termination, and collect their termination status. These calls help the controller coordinate with workers and prevent terminated child processes from remaining as zombies.

## 2.2 Existing Methods / Algorithms

Different mechanisms can be used to communicate with and control processes in an operating system. The choice depends on the amount of data exchanged, synchronization requirements, complexity, and application needs.

### 2.2.1 Manual Process Management

Users can control processes through terminal commands or system utilities. These commands can inspect processes, stop execution, resume stopped processes, or terminate applications. This approach is useful for basic administration but is less convenient when multiple worker processes must be controlled automatically by one application.

### 2.2.2 Polling-Based Process Monitoring

Polling repeatedly checks the state of another process or resource at regular intervals. Although straightforward, polling may consume unnecessary CPU time and introduce delays between an event and its detection.

### 2.2.3 Pipe-Based Communication

Pipes transfer data between processes. They are commonly used between related processes, while named pipes can support communication between processes that are not directly related. Pipes are suitable for streams of data, but the application must manage data transfer and interpret received information.

### 2.2.4 Shared Memory and Message Queues

Shared memory allows multiple processes to access a common memory region and can provide efficient data exchange, but synchronization is required to avoid conflicting access. Message queues allow processes to exchange structured messages through an operating-system-managed queue. Both can be effective IPC solutions but may introduce additional setup and management requirements.

### 2.2.5 Signal-Based Process Control

Signal-based process control uses signals to notify processes about events and request specific actions. It is suitable for lightweight notifications and process-control operations. Traditional signals are not intended for transferring large amounts of application data, and ordinary signals may not preserve every repeated occurrence as a separate queued event. This project uses signals because its main purpose is to demonstrate process control and event notification.

## 2.3 Proposed Approach

The proposed system implements a controller-based process-management mechanism using Linux signals and system calls written in C. The controller creates worker processes, communicates with them using signals, and monitors their execution.

### 2.3.1 Controller Process

The controller creates worker processes using fork(), stores their process identifiers, initiates control operations, and sends signals to selected workers using kill(). It also monitors process status and waits for child processes to terminate.

### 2.3.2 Worker Processes

Worker processes are created by the controller and execute independently. Each worker identifies itself using its process ID and waits for relevant signals or performs its assigned activity. When a supported signal is received, the worker responds according to the implemented signal-handling logic.

### 2.3.3 Signal Communication Module

The signal communication module allows the controller to send control requests to worker processes. kill() delivers signals to the appropriate process. Signal handlers configured with

sigaction() allow workers to respond to supported user-defined signals. SIGSTOP, SIGCONT, and SIGTERM can also be used for process control, subject to their operating-system behavior.

### 2.3.4 Process Monitoring Module

The monitoring module tracks worker PIDs and observes their execution status. It helps the controller determine whether a worker is active, stopped, or terminated using process-state checks and child-process status information. wait() or waitpid() collects termination status and supports orderly cleanup.

### 2.3.5 Working Procedure

The controller process starts execution.
The controller creates worker processes using fork().
Each worker obtains its process ID and initializes the required signal-handling behavior.
The controller initiates or accepts a process-control operation.
The controller sends the appropriate signal to the selected worker using kill().
The worker responds according to the signal received and its configured behavior.
The controller monitors the workers and coordinates their execution.
When a worker terminates, the controller collects its termination status using wait() or waitpid().
The system completes execution after the required processes have been handled.

## 2.4 Advantages of the Proposed Approach

### 2.4.1 Simple Communication Mechanism

Signals provide a straightforward way to notify processes about events and request specific actions.

### 2.4.2 Efficient Process Control

The controller can direct signals to selected workers without continuously polling them for every control event.

### 2.4.3 Modular Architecture

Separating the controller from worker processes makes the application easier to understand, implement, test, and maintain.

### 2.4.4 Improved Process Coordination

Process identifiers, signal delivery, and child-process waiting mechanisms help coordinate worker execution and termination.

### 2.4.5 Practical Demonstration of Linux System Calls

The project provides hands-on experience with fork(), kill(), getpid(), sigaction(), wait(), and waitpid(), as applicable.

### 2.4.6 Resource Management

Proper synchronization and cleanup help prevent terminated child processes from remaining as zombies.

### 2.4.7 Educational Value

The project connects operating-system theory with practical system programming and helps students understand process creation, signal delivery, process states, IPC, signal handling, and synchronization.

# CHAPTER – 3

## SYSTEM REQUIREMENTS

### 3.1 Hardware Requirements

The project does not require specialized hardware. A standard computer capable of running Ubuntu or a Linux environment is sufficient.

Computer or laptop with a modern multi-core processor.

At least 4 GB RAM recommended for comfortable development.

Sufficient free disk space for Ubuntu/WSL, source files, and development tools.

Keyboard and display for terminal-based compilation and testing.

### 3.2 Software Requirements

Ubuntu Linux or Windows Subsystem for Linux (WSL) with Ubuntu.

C programming language.

GNU Compiler Collection (GCC).

GNU nano, Vim, or another text editor.

Linux terminal and standard process-management utilities.

### 3.3 Operating System Environment – Ubuntu / WSL

Ubuntu provides a Linux environment with support for POSIX process and signal APIs. The project can be developed and executed in Ubuntu installed directly on a computer or through WSL on Windows. The terminal is used to create source files, compile the program, run the executable, and observe process behavior.

### 3.4 Development Tools and Compilers

GCC is used to compile the C source code into an executable program. A text editor such as nano can be used to create and modify source files. Linux terminal commands support compilation, execution, and testing. The exact commands depend on the source filename used by the project.

# CHAPTER – 4

## TECHNOLOGY AND TOOLS USED

### 4.1 C Programming

C is a general-purpose programming language widely used for system programming. It provides access to low-level operating system interfaces through standard libraries and system calls. In this project, C is used to create processes, send signals, handle events, and coordinate process termination.

### 4.2 GCC Compiler

The GNU Compiler Collection (GCC) compiles C source code into an executable program. A typical compilation command is gcc source.c -o program. Compiler warnings and error messages help identify syntax errors and other implementation issues.

### 4.3 Linux System Calls

Linux system calls provide interfaces through which programs request operating-system services. The system calls and functions relevant to this project include:
fork(): creates a child process.
kill(): sends a signal to a process or process group.
getpid(): returns the calling process ID.
getppid(): returns the parent process ID.
sigaction(): configures handling for supported signals.
wait() and waitpid(): wait for child-process state changes and collect status.

### 4.4 POSIX Signal Handling and Synchronization

POSIX signal APIs support configuring signal handlers and managing signal masks. The program must account for asynchronous signal delivery and use safe handler behavior. The wait() and waitpid() interfaces support synchronization between a parent and its child processes.

### 4.5 Linux Commands and Utilities

nano: edits C source files in the terminal.
gcc: compiles the source code.
./program: runs the compiled executable.
ps: displays process information.
kill: sends signals from the command line when permitted.
man: opens manual pages for commands and system interfaces.

### 4.6 Other Tools / Libraries Used

The project uses standard C and POSIX/Linux headers required for process creation, signals, and waiting for child processes. Common headers may include stdio.h, stdlib.h, unistd.h, signal.h, and sys/wait.h, depending on the implementation.

# CHAPTER – 5

## SYSTEM ARCHITECTURE AND DESIGN

### 5.1 Overall System Architecture

The system follows a controller-worker architecture. The controller is responsible for creating worker processes, issuing signals, and coordinating cleanup. Worker processes execute independently and respond to signals according to the behavior implemented in the C program.

**CONTROLLER PROCESS**

Creates workers using fork() | Tracks worker PIDs

Sends signals using kill() and monitors status

↓ ↓

**WORKER PROCESS 1 WORKER PROCESS 2**

Signal handling and assigned activity in each worker

↓

wait() / waitpid() for child-process status and cleanup

### 5.2 System Workflow

Start the controller program.

Create worker processes.

Initialize each worker's signal behavior.

Select or initiate a control action.

Send a signal to the target worker.

Worker responds according to the signal and implementation.

Monitor worker state and collect termination status.

Finish after the required workers are handled.

### 5.3 Flow of Control

The controller begins the process lifecycle and records the process IDs returned through process creation. It sends signals to the appropriate process when a control action is required. Workers respond to the signals they are configured to handle, while signals with default operating-system behavior follow their defined semantics. When a child process terminates, the parent collects its status and performs cleanup.

### 5.4 Module Description

**Controller Module**

Creates and manages worker processes, tracks PIDs, sends signals, and coordinates shutdown.

**Worker Module**

Performs the assigned worker activity and responds to supported signals.

**Signal Handling Module**

Configures and implements responses to selected signals.

**Monitoring and Cleanup Module**
Checks process status and collects child-process termination information.

### 5.5 Input and Output Design

Inputs may include the selected worker process and the control operation supported by the program. Depending on the implementation, the controller may accept commands through the terminal or use a predefined sequence of operations.
Outputs may include process IDs, messages indicating signal delivery, worker responses, process-state information, and termination status. The exact output depends on the implemented program and should be documented using actual execution screenshots.

# CHAPTER – 6

## IMPLEMENTATION

### 6.1 Module-wise Implementation

The program is implemented in C and compiled with GCC in Ubuntu/Linux. The implementation is organized around process creation, signal delivery, worker response, monitoring, and cleanup. The following descriptions should be checked against the final source code before submission.

### 6.2 Algorithm / Pseudocode

**Algorithm: Linux Signal-Based Process Control**
Begin execution in the controller.
Create worker processes using fork() and check for errors.
In each worker, initialize the required signal-handling behavior.
In the controller, record each worker PID.
For each requested operation, choose the target worker and signal.
Send the signal and handle any reported errors.
Monitor or wait for child-process state changes as required.
Collect terminated child-process status and release resources.
End execution.

### 6.3 Core C Implementation

The source code should include the required headers and check return values from system calls. fork() should be checked for failure; kill() should be checked to detect errors such as an invalid PID or insufficient permissions. Signal handlers should avoid unsafe library operations. The final report should insert the exact source code used in the working demonstration.

### 6.4 Linux System Calls / APIs Used

fork() creates worker processes. kill() sends a selected signal to a process. getpid() and getppid() provide process identifiers. sigaction() configures handlers for supported signals.

wait() and waitpid() collect child-process state information and help the parent clean up terminated children.

### 6.5 Process Implementation

The controller creates workers and distinguishes parent and child execution using the return value of fork(). Each worker follows its own execution path, while the parent maintains information needed to manage the workers. The number of workers and their tasks should match the actual source code.

### 6.6 IPC / Synchronization Implementation

Signals are used for event notification and control. The parent and child processes coordinate through signal delivery and child-process waiting. Signals do not provide a general data stream; if the application needs to transfer detailed messages, another IPC mechanism would be required.

### 6.7 Memory Management Implementation

The operating system allocates a separate process address space for each child process created by fork(), with implementation optimizations such as copy-on-write. The application should release any resources it explicitly allocates and reap terminated child processes using wait() or waitpid().

### 6.8 Optional User Interface Implementation

The project can use a terminal-based interface to display process IDs, available operations, and status messages. Any menu or commands described here must match the actual implementation. If no interactive menu is implemented, this section should be treated as a description of the terminal interface rather than a separate graphical user interface.

# CHAPTER – 7

## EXECUTION AND DEMONSTRATION

### 7.1 Compilation in Ubuntu

Open the Ubuntu terminal and move to the directory containing the C source file. Compile the source file using GCC. For example, if the source file is named question.c, the command is:
gcc question.c -o question
If the program uses additional source files, include them in the compilation command. Fix compiler errors before proceeding.

### 7.2 Execution Commands

Run the compiled program from the directory containing the executable:
./question
Use the actual executable name if it differs. If process inspection is part of the demonstration, use suitable terminal utilities such as ps in a separate terminal.

### 7.3 Sample Inputs and Outputs

Insert the actual output produced by the final program. The output may show controller and worker process IDs, signal operations, worker responses, and child-process termination status. Do not use illustrative output as proof of successful execution.

### 7.4 Screenshots of Execution

Include clear screenshots showing the source code, successful GCC compilation, program execution, signal-based control operations, and final cleanup. Ensure that screenshots correspond to the same source code included in the report.

### 7.5 Demonstration of OS Concepts

Process creation through fork().
Process identification through process IDs.
Signal delivery using kill().
Signal handling for signals configured by the program.
Process-state changes where demonstrated.
Child-process synchronization and cleanup using wait() or waitpid().

# CHAPTER – 8

## CONCLUSION

### 8.1 Summary of the Project

The Linux Signal-Based Process Communication and Control System demonstrates how Linux processes can be created, notified, and controlled using system calls and signals. A controller process manages worker processes and coordinates their execution.

### 8.2 Objectives Achieved

The project is intended to demonstrate process creation, signal delivery, process identification, signal handling, and child-process synchronization. The objectives achieved should be confirmed against the completed implementation and test results.

### 8.3 Results Achieved

The final results should be reported from actual tests in Ubuntu/Linux. Include evidence that the program compiles, creates the expected processes, sends the intended signals, produces the expected worker responses, and cleans up child processes correctly.

### 8.4 Skills and Learning Outcomes

Through this project, students gain practical experience in C system programming, Linux commands, process management, signal handling, debugging, and synchronization. The project strengthens understanding of how operating-system concepts are applied in a working program.

# CHAPTER – 9

**REFERENCES**

**9.1 Books**

Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, Operating System Concepts. Use the edition available to the team.

W. Richard Stevens and Stephen A. Rago, Advanced Programming in the UNIX Environment. Use the edition consulted.

**9.2 Research Papers**

Add any research papers actually consulted for this project. Do not list papers that were not reviewed.

**9.3 Linux / GNU Documentation**

Linux manual pages: fork(2), kill(2), getpid(2), sigaction(2), wait(2), and waitpid(2).

GNU Compiler Collection (GCC) documentation.

**9.4 Online Resources**

Ubuntu documentation: https://help.ubuntu.com/

The Linux man-pages project: https://man7.org/linux/man-pages/